



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

COMMON COURSE ASSIGNMENT

Smart Grid Municipal Energy Analytics System and Peak Load Predictor

CSA0812 PYTHON PROGRAMMING

to the award of the degree of

BACHELOR OF Technology

Submitted by

D.Dileep Kumar-192424281

Under the Supervision of

Dr.Prabaharan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

January 2026

Assignment Information

Department:	Computer Science and Engineering
Programme:	B.Tech / B.E.
Course Code & Course Name:	CSA0812 / Python Programming
Academic Year / Batch:	2026–2027 / MS1 July-Sep 2026
Faculty Name:	Dr. Prabakaran J
Assignment Title:	Smart Grid Municipal Energy Analytics System and Peak Load Predictor
Date of Issue:	01/09/2026
Date of Submission:	02/09/2026
Maximum Marks:	100
Course Outcome(s) – CO:	CO3: Design modular Python systems using multi-dimensional dictionary datasets, string array operations, and custom function definitions to solve numerical optimization problems.
Bloom's Taxonomy Level:	Apply (L3) & Analyze (L4)
SDG Mapping:	SDG 7: Affordable and Clean Energy & SDG 13: Climate Action
Industry / Societal Relevance:	High-fluctuation smart microgrids require precise local load forecasting. Sudden demand peaks overwhelm substations, forcing non-renewable generator kick-ins or brownouts. Building programmatic prediction tools allows municipal regions to accurately map consumption trends, automatically flag anomalous power drainage, and run dynamic pricing adjustments to stabilize municipal power distribution.

Assignment Problem / Challenge

Faculty challenge requirements fulfilled:

- Students will apply data structures (dictionaries, lists), file parsing, string processing, and exception handling instead of reproducing syntax theory.
- Students must identify and filter malformed records, analyze environmental safety metrics, and interpret threshold breaches.
- Students will implement alternative validation or cleaning pathways and use Python as a modern software engineering tool.

Problem Statement

An industrial manufacturing facility utilizes automated air-quality and temperature sensors throughout its assembly line. The sensor hardware exports a daily raw log file named `factory_sensor_logs.txt`. However, due to intermittent network interference and fluctuating power grids, the generated log data contains multiple corrupted entries, missing telemetry coordinates, an out-of-order timestamps.

You are tasked with designing and implementing a robust Python application that parses this raw file, isolates corrupt lines without crashing, tracks moving averages for air quality particles (PM2.5) and ambient temperatures, triggers alerts if threshold limits are breached, and exports a clean, structured anomaly summary file named `environmental alert report.csv`

Requirements and Constraints

☐ Functional Requirements

- Read hourly smart-grid telemetry data from a #-delimited text file.
- Validate each record and reject corrupted or incomplete data.
- Identify valid sectors: RES, COM, and IND.
- Calculate sector-wise total load, average load, peak load, and billing.
- Calculate the demand-surge penalty multiplier for each sector.
- Classify sectors as NORMAL, ELEVATED, or CRITICAL.
- Generate a prioritized load-balancing operations plan.
- Display the analysis through a clean console dashboard.
- Export the final balancing plan and analysis to a text file.

☐ Technical Requirements

- Use Python standard-library modules only.
- Use functions and a multi-dimensional dictionary for data processing.
- Process the telemetry log in a single linear pass $O(n)$.
- Use string operations such as `split()` and `strip()` for input processing.
- Handle invalid input using appropriate exception handling.
- Ensure one corrupted record does not affect valid records from other sectors.

□ **Data Validation Requirements**

- Detect malformed records with an incorrect number of fields.
- Detect unknown or invalid sector codes.
- Detect missing timestamps or numeric fields.
- Safely convert load and tariff values into numerical types.
- Reject non-numeric, negative load, and invalid tariff values.
- Record the reason for every rejected telemetry record.

□ **Output Requirements**

- Display the number of valid and rejected records.
- Display rejection reasons.
- Display sector-wise load statistics.
- Display surge multiplier and priority for each sector.
- Display recommended balancing actions.
- Export the complete analysis to `balancing_operations_summary.txt`

1. Problem Understanding and Formulation

What is the problem?

A municipal microgrid hub records hourly power draw for three city sectors — Residential, Commercial, and Heavy Industrial — as raw '#'-delimited text logs (Sector_Code # Timestamp # Grid_Load_kWh # Current_Tariff_Rate). Because the metering hardware suffers calibration drift, the exported stream regularly contains missing numeric fields, unrecognised sector codes, and physically impossible negative load readings. The task is to build a functions-driven Python

program that discards unreliable telemetry, tallies genuine consumption by sector, flags sectors that are drawing power beyond their safe threshold, and writes a prioritized balancing plan that a grid operator can act on.

What are the expected outcomes?

- A working Python script that reads the raw log and separates valid readings from corrupted ones, with the rejection reason recorded.
- A sector-wise aggregate of total, average, and peak load, plus estimated billing.
- A demand-surge penalty multiplier and priority label (NORMAL / ELEVATED / CRITICAL) for every sector.
- A console dashboard and an exported text file containing a prioritized balancing operations plan.

What information/data is available?

The only input is the raw hourly telemetry stream. Each line, when well-formed, supplies four fields: a three-letter sector code (RES, COM, IND), a timestamp, the hourly load in kWh, and the tariff rate applicable to that hour (Rs./kWh). No external weather, seasonal, or historical dataset is supplied, so the program has to work from this single stream alone.

What assumptions are made?

- Only three sector codes are valid: RES (Residential), COM (Commercial), IND (Heavy Industrial). Any other code is treated as a misaligned key, not a new sector.
- Each line represents exactly one hourly reading for one sector; there are no multi-hour or multi-sector lines.
- A record with a missing field, a non-numeric value, or a negative load/tariff is a genuine sensor fault and is discarded outright rather than repaired, since silently estimating a replacement value could hide a real equipment failure.
- Tariff rate is always a positive number expressed in Rupees per kWh.

What constraints must be satisfied?

- Functional: filter corrupted records, aggregate by sector, compute a surge multiplier, export a balancing plan — all in one interactive run.
- Performance: the log must be processed in a single linear pass ($O(n)$) so the tool stays usable as the log grows through the day.
- Technical: implementation restricted to the Python standard library, in line with a lightweight utility-operations deployment.

- Reliability: a single corrupted line must never crash the program or corrupt the aggregates of other sectors.
- Sustainability: the balancing recommendations should actively discourage peak draw that would otherwise be met by non-renewable backup generation (SDG 7 and SDG 13).

2. Application of Course Knowledge

Multi-dimensional dictionary datasets

Two static lookup dictionaries and one computed nested dictionary drive the whole program. SECTOR_LOOKUP maps a raw three-letter code to its full sector name, and SURGE_THRESHOLD_KWH maps each sector name to its safe hourly load ceiling. The core dataset, sector_stats, is a dictionary of dictionaries — sector_stats[sector_name] holds reading_count, total_load_kwh, peak_load_kwh, total_billing, and avg_load_kwh — which is exactly the multi-dimensional structure the course outcome asks for.

String array operations

Every raw line is split on the '#' delimiter with str.split("#"), each resulting field is cleaned with str.strip(), and rejection-reason keys are turned into readable labels with str.replace("_", " ").title() when the dashboard is printed. Field-count validation (len(fields) != 4) and type coercion via float() together form the string-to-numeric pipeline the course covers.

Custom function definitions

The program is decomposed into eight single-purpose functions — parse_record, load_telemetry, aggregate_by_sector, compute_surge_multiplier, classify_priority, generate_balancing_plan, print_dashboard, and export_summary — so that each numerical or logical step is independently testable and reusable, matching the modular design the CO expects.

Mathematical model — averaging

For a sector with n valid readings $L_1 \dots L_n$, the average hourly load is $\text{avg} = (L_1 + L_2 + \dots + L_n) / n$. This is computed incrementally: total_load_kwh accumulates the sum during the single pass over records, and avg_load_kwh is derived once at the end (total_load_kwh / reading_count), avoiding a second pass over the data.

Mathematical model — surge penalty multiplier

Let T be a sector's safety threshold and L its average hourly load. The multiplier is a piecewise linear function:

```
multiplier(L) = 1.0                if L <= T
multiplier(L) = min(1.0 + (L - T)/T, 2.5)  if L > T
```

Worked example using the actual test run: Heavy Industrial has $T = 3300$ kWh and an observed average load of 3817.87 kWh. Excess ratio = $(3817.87 - 3300) / 3300 = 517.87 / 3300 = 0.1569$. Multiplier = $1 + 0.1569 = 1.1569$, rounded to 1.16, which is below the 2.5 cap and is reported as-is. This multiplier is a direct measure of how far a sector is running above its rated safe capacity, expressed as a fraction of that capacity.

Nested conditional logic

classify_priority applies a three-tier nested condition on the multiplier: multiplier ≥ 1.5 is CRITICAL, $1.0 < \text{multiplier} < 1.5$ is ELEVATED, and multiplier $= 1.0$ is NORMAL. generate_balancing_plan then branches on this label to select one of three distinct recommended actions, which is the multi-tier nested conditional expression the assignment calls for.

3. Solution / Design / Methodology

System architecture (pipeline)

The program follows a five-stage linear pipeline: (1) Ingest — read the raw log file line by line; (2) Validate/Filter — parse and reject malformed records, logging the reason; (3) Aggregate — accumulate totals, peaks, and averages into the sector_stats dictionary; (4) Surge Analysis — compute a penalty multiplier and priority per sector; (5) Plan & Export — sort sectors by urgency, attach a recommended action to each, print the dashboard, and write the summary file. Each stage consumes only the output of the previous one, so a fault in one sector's data can never contaminate another sector's statistics.

Pseudocode

```

READ each line of the log file
FOR each line:
    SPLIT on '#'
    IF field count != 4 OR sector code unknown OR any numeric
        field missing/non-numeric/negative:
            REJECT, record the reason
    ELSE:
        ADD record to valid list
FOR each sector:
    total, count, peak <- accumulate from valid records
    avg <- total / count
    multiplier <- surge formula using avg and threshold
    priority <- NORMAL / ELEVATED / CRITICAL
SORT sectors by multiplier, descending
PRINT dashboard; WRITE balancing plan to output file

```

Alternative designs considered

Two design choices were evaluated before settling on the current approach. First, corrupted numeric fields could have been repaired by interpolating from neighbouring hours instead of being discarded; this was rejected because it risks masking a genuine sensor fault behind a plausible-looking estimate, which conflicts with the reliability and safety requirements of a grid-monitoring tool. Second, the surge threshold could have been computed dynamically from the sector's own historical mean and standard deviation instead of a fixed constant; this was set aside for the current scope because it needs a longer historical window than the single-day log provides, but it is noted as a natural extension in Section 8.

4. Use of Modern Tools

- Python 3.12 (standard library only — os, datetime), run from a terminal / VS Code.
- The script was executed directly (python3 smart_grid_analytics.py) against a 25-line sample log seeded with deliberate faults, to exercise every rejection path.
- Version control: the working script and sample log were tracked as a local Git repository (git init, git add, git commit) to preserve an edit history, as expected for a functions-driven deliverable.
- Output verification was done by redirecting console output to a file and inspecting the exported balancing_operations_summary.txt for consistency with the on-screen dashboard.
- The complete source code, raw input log, full console output, and the exported summary file are reproduced in full in Appendices A-D as direct evidence of tool usage and execution.

5. Results and Validation

Test dataset

A 25-line sample log was constructed with 19 well-formed readings and 6 deliberately corrupted lines covering every rejection category: a missing numeric field, a non-numeric value, an unrecognised sector code, a malformed field count, and two negative load values. This was designed specifically to validate the filtering logic end to end.

Rejected-record breakdown

Rejection Reason	Count
Negative or invalid value	2
Missing numeric field	1
Misaligned sector key	1
Malformed field count	1
Non-numeric value	1

Sector-wise aggregates (19 valid records)

Sector	Readings	Total (kWh)	Avg (kWh)	Peak (kWh)	Billing (Rs.)
Residential	6	2840.00	473.33	601.40	17893.42
Commercial	7	12354.80	1764.97	1955.60	90685.58
Heavy Industrial	6	22907.20	3817.87	4320.70	138950.89

Prioritized balancing operations plan (as exported)

Rank	Sector	Multiplier	Priority	Recommended Action
1	Heavy Industrial	x1.16	ELEVATED	Dynamic tariff surcharge + demand-response advisory
2	Commercial	x1.10	ELEVATED	Dynamic tariff surcharge + demand-response advisory
3	Residential	x1.00	NORMAL	Maintain standard distribution schedule

Validation

- Record accounting checks out: 19 valid + 6 rejected = 25 raw lines, matching the input file exactly.
- Manual spot check: Residential total = $412.5+388.2+455.7+421.9+560.3+601.4 = 2840.0$ kWh, matching the program's output.
- Sector ranking is correct: Heavy Industrial (16% over threshold) is placed above Commercial (10% over threshold), which is placed above Residential (under threshold), confirming the descending sort by multiplier.

- Re-running the script on the same file reproduces identical output, confirming the pipeline is deterministic.

6. Analysis and Engineering Decision

Fixed threshold vs statistical surge detection

A fixed kWh threshold per sector was chosen over a statistical method (e.g., flagging readings beyond the sector's own mean plus two standard deviations). The fixed threshold is deterministic and auditable — a regulator or operator can verify a surge call against a published, unchanging number — whereas a statistical threshold would shift as more data arrives and would need a longer historical window than a single day's log provides. The trade-off is that a fixed threshold does not automatically adapt to seasonal or weekday/weekend load patterns, which is flagged as a limitation in Section 8.

Discard vs repair of corrupted records

Discarding a corrupted record loses that hour's data point entirely, slightly understating total consumption for the sector in that hour. Repairing it (e.g., by interpolation) would preserve continuity but risks reporting a fabricated number as if it were measured, which is unacceptable for a system that can trigger automatic load-shedding. Given the safety-critical nature of the CRITICAL action path, discard was chosen as the more defensible option.

Multiplier cap at 2.5

Capping the multiplier prevents a single extreme outlier reading from producing an unrealistic penalty or surcharge. Since the multiplier feeds directly into the recommended action, an uncapped value could recommend an overly aggressive load-shed even from one bad hour; capping keeps the response proportionate.

Memory usage of the nested dictionary

sector_stats holds a fixed five-key dictionary for each of the three sectors, so its memory footprint is $O(1)$ with respect to the number of log lines — it does not grow as the log grows, only the intermediate valid_records list does. This means the aggregation stage scales to a much longer log (e.g., a full month of hourly readings) without the summary structure itself becoming a memory concern; the only linear-memory component is the raw record list retained before aggregation, which could be streamed and discarded per-line in a future version if very large logs are expected.

7. Broader Considerations

- Sustainability / SDG 7 & SDG 13: by surfacing which sector is drawing power closest to its safe ceiling, the tool lets an operator apply demand-response pricing before a non-renewable peaker generator has to be switched on, directly supporting affordable clean energy and climate action goals.
- Safety: the CRITICAL action path (automatic load-shedding on non-essential feeders) is designed to prevent transformer or substation overload before it becomes a physical safety incident.
- Ethics and fairness: the dynamic tariff surcharge is applied uniformly at the sector level rather than targeting individual households or businesses, and the program never stores or processes any personally identifiable consumer data — only aggregated sector totals.
- Economics: surcharge-based demand response is comparatively cheap to implement (a pricing signal) compared to physical infrastructure upgrades, though it must be capped (see Section 6) to avoid unfair billing shocks.
- Professional responsibility: rejecting rather than silently repairing corrupted telemetry reflects an engineering duty to report equipment faults honestly rather than paper over them with plausible-looking substitute data.

8. Conclusion

The assignment required a functions-driven Python program that turns a noisy municipal microgrid telemetry stream into an actionable balancing plan. The delivered solution parses and validates every record against four independent fault conditions, aggregates clean readings into a multi-dimensional sector dictionary, applies a threshold-based surge multiplier, and exports a prioritized action plan. On the 25-line test log, it correctly rejected all six seeded faults, correctly computed sector averages and totals, and correctly ranked Heavy Industrial above Commercial above Residential by demand-surge severity — meeting every functional requirement set out in Section 1. The main limitation is that the surge threshold is a fixed constant rather than one that adapts to season, day-of-week, or weather; the tool is also validated only against a single synthetic day of data rather than a longer real-world stream. A natural improvement would be to replace the fixed threshold with a rolling statistical baseline and to extend the exported plan with a short-term load forecast rather than only a same-hour surge flag.

9. Student Reflection

Beyond the syntax covered in class, this assignment made the gap between a 'clean CSV' and a real operational data stream very concrete — a real telemetry feed does not arrive validated, and the filtering logic ends up being as much of the program as the analytics itself. Designing the

nested `sector_stats` dictionary also clarified why multi-dimensional dictionaries are preferred over parallel lists for this kind of grouped-aggregation problem: every sector's five statistics stay together under one key instead of having to be tracked across five separate arrays kept in sync by index.

Given more time, the threshold-based surge detection would be replaced with a short-horizon forecasting model (even a simple linear regression over the last few hours per sector) so the plan could flag a surge before it happens rather than only after it is already occurring. I would also add automated unit tests for `parse_record` against a larger set of malformed-input cases, and export the summary as CSV/JSON in addition to plain text so the balancing plan could feed directly into a downstream dashboard.

10. References

- Python Software Foundation. Python 3.12 Documentation — Built-in Types, string methods, and the `os` and `datetime` modules. <https://docs.python.org/3/>
- United Nations. Sustainable Development Goal 7: Affordable and Clean Energy. <https://sdgs.un.org/goals/goal7>
- United Nations. Sustainable Development Goal 13: Climate Action. <https://sdgs.un.org/goals/goal13>
- Course material and problem statement supplied for CSA0812 — Python Programming, Common Course Assignment: Smart Grid Energy Analytics, SIMATS Engineering, MS1 July-Sep 2026.
- AI-assisted tool disclosure: Claude (Anthropic) was used to help structure the modular function design and draft this report's prose; all code was tested and verified by the student before submission, and the sample dataset, calculations, and analysis were independently checked against the program's own output.

Appendix A: Complete Python Source Code (smart_grid_analytics.py)

The full, unmodified source code of the program discussed in Sections 2-5 is reproduced below for reference and evaluation.

```
"""
Smart Grid Municipal Energy Analytics System and Peak Load Predictor
Course: CSA0812 - Python Programming
Assignment: Smart Grid Energy Analytics

This module reads raw hourly telemetry logs from a municipal microgrid,
filters corrupted records, aggregates power draw by city sector, computes
a demand surge penalty multiplier per sector, and exports a prioritized
load-balancing operations plan.
"""

import os
from datetime import datetime

# -----
# 1. STATIC CONFIGURATION (dynamic lookup maps)
# -----

# Maps the 3-letter sector code found in the raw stream to a human-readable
# sector name. Any code not present here is treated as a misaligned key.
SECTOR_LOOKUP = {
    "RES": "Residential",
    "COM": "Commercial",
    "IND": "Heavy_Industrial",
}

# Hourly load threshold (kWh) above which a sector is considered to be in
# a demand-surge condition. These values stand in for substation-rated
# safe-carry capacity per sector.
SURGE_THRESHOLD_KWH = {
    "Residential": 550.0,
    "Commercial": 1600.0,
    "Heavy_Industrial": 3300.0,
}

# Multiplier is capped so that a single extreme outlier reading cannot
# push a sector's penalty beyond a realistic operational ceiling.
MAX_PENALTY_MULTIPLIER = 2.5

# -----
# 2. RECORD PARSING AND VALIDATION
# -----
```

```

def parse_record(raw_line):
    """
    Parses a single '#'-delimited telemetry line into a validated record.

    Expected format:
        Sector_Code # Timestamp # Grid_Load_kWh # Current_Tariff_Rate

    Returns a dict on success, or None (with a reason string) on failure.
    """
    fields = raw_line.strip().split("#")

    if len(fields) != 4:
        return None, "malformed_field_count"

    sector_code, timestamp, load_raw, tariff_raw = (f.strip() for f in
fields)

    if sector_code not in SECTOR_LOOKUP:
        return None, "misaligned_sector_key"

    if not timestamp:
        return None, "missing_timestamp"

    if load_raw == "" or tariff_raw == "":
        return None, "missing_numeric_field"

    try:
        load_kwh = float(load_raw)
        tariff_rate = float(tariff_raw)
    except ValueError:
        return None, "non_numeric_value"

    if load_kwh < 0 or tariff_rate <= 0:
        return None, "negative_or_invalid_value"

    record = {
        "sector": SECTOR_LOOKUP[sector_code],
        "timestamp": timestamp,
        "load_kwh": load_kwh,
        "tariff_rate": tariff_rate,
    }
    return record, "ok"

def load_telemetry(filepath):
    """
    Reads the raw log file and separates valid records from rejected ones.
    Returns (valid_records, rejection_log) where rejection_log is a dict
    of reason -> count.
    """
    valid_records = []

```

```

rejection_log = {}

with open(filepath, "r") as log_file:
    for line_number, raw_line in enumerate(log_file, start=1):
        if not raw_line.strip():
            continue
        record, status = parse_record(raw_line)
        if status == "ok":
            valid_records.append(record)
        else:
            rejection_log[status] = rejection_log.get(status, 0) + 1

return valid_records, rejection_log

# -----
# 3. AGGREGATION (multi-dimensional dictionary dataset)
# -----

def aggregate_by_sector(records):
    """
    Builds a multi-dimensional dictionary keyed by sector name, holding
    cumulative and derived statistics for each sector.
    """
    sector_stats = {}

    for sector_name in SECTOR_LOOKUP.values():
        sector_stats[sector_name] = {
            "reading_count": 0,
            "total_load_kwh": 0.0,
            "peak_load_kwh": 0.0,
            "total_billing": 0.0,
        }

    for record in records:
        stats = sector_stats[record["sector"]]
        stats["reading_count"] += 1
        stats["total_load_kwh"] += record["load_kwh"]
        stats["total_billing"] += record["load_kwh"] * record["tariff_rate"]
        if record["load_kwh"] > stats["peak_load_kwh"]:
            stats["peak_load_kwh"] = record["load_kwh"]

    for sector_name, stats in sector_stats.items():
        if stats["reading_count"] > 0:
            stats["avg_load_kwh"] = stats["total_load_kwh"] /
stats["reading_count"]
        else:
            stats["avg_load_kwh"] = 0.0

    return sector_stats

```

```

# -----
# 4. SURGE PENALTY MULTIPLIER
# -----

def compute_surge_multiplier(sector_name, avg_load_kwh):
    """
    Computes a demand-surge penalty multiplier for a sector based on how
    far its average hourly load exceeds the sector's safety threshold.

    multiplier = 1.0 if avg_load <=
threshold
    multiplier = 1.0 + (avg_load - threshold)/threshold otherwise, capped
    """
    threshold = SURGE_THRESHOLD_KWH[sector_name]

    if avg_load_kwh <= threshold:
        return 1.0

    excess_ratio = (avg_load_kwh - threshold) / threshold
    multiplier = 1.0 + excess_ratio
    return round(min(multiplier, MAX_PENALTY_MULTIPLIER), 2)

def classify_priority(multiplier):
    """Maps a numeric multiplier to an operational priority label."""
    if multiplier >= 1.5:
        return "CRITICAL"
    elif multiplier > 1.0:
        return "ELEVATED"
    return "NORMAL"

# -----
# 5. BALANCING OPERATIONS PLAN
# -----

def generate_balancing_plan(sector_stats):
    """
    Produces a prioritized list of balancing actions, one entry per sector,
    sorted from most to least urgent (highest multiplier first).
    """
    plan = []

    for sector_name, stats in sector_stats.items():
        multiplier = compute_surge_multiplier(sector_name,
stats["avg_load_kwh"])
        priority = classify_priority(multiplier)

        if priority == "CRITICAL":
            action = (

```



```

        f"Trigger automatic load-shedding on non-essential
{sector_name} "
        f"feeders and dispatch reserve generation capacity
immediately."
    )
    elif priority == "ELEVATED":
        action = (
            f"Apply dynamic tariff surcharge to {sector_name} consumers
and "
            f"issue an early demand-response advisory."
        )
    else:
        action = f"Maintain standard distribution schedule for
{sector_name}."

    plan.append({
        "sector": sector_name,
        "avg_load_kwh": round(stats["avg_load_kwh"], 2),
        "threshold_kwh": SURGE_THRESHOLD_KWH[sector_name],
        "multiplier": multiplier,
        "priority": priority,
        "action": action,
    })

    plan.sort(key=lambda entry: entry["multiplier"], reverse=True)
    return plan

# -----
# 6. CONSOLE DASHBOARD
# -----

def print_dashboard(sector_stats, plan, rejection_log, total_valid):
    print("\n" + "=" * 72)
    print(" SMART GRID MUNICIPAL ENERGY ANALYTICS - PEAK LOAD DASHBOARD")
    print("=" * 72)

    print(f"\nValid records processed : {total_valid}")
    total_rejected = sum(rejection_log.values())
    print(f"Corrupted records rejected: {total_rejected}")
    if rejection_log:
        for reason, count in rejection_log.items():
            print(f"    - {reason.replace('_', ' ').title():<28}: {count}")

    print("\n" + "-" * 72)
    print(f"{'SECTOR':<18}{'READINGS':<10}{'TOTAL kWh':<14}{'AVG kWh':<12}{'PEAK kWh':<12}")
    print("-" * 72)
    for sector_name, stats in sector_stats.items():
        print(f"{'sector_name':<18}{'stats['reading_count']':<10}")

```

```

f"{stats['total_load_kwh']:<14.2f}{stats['avg_load_kwh']:<12.2f}"
    f"{stats['peak_load_kwh']:<12.2f}")

print("\n" + "-" * 72)
print("PRIORITIZED BALANCING OPERATIONS PLAN")
print("-" * 72)
for rank, entry in enumerate(plan, start=1):
    print(f"\n[{rank}] Sector      : {entry['sector']}")
    print(f"    Avg Load      : {entry['avg_load_kwh']} kWh "
          f"(Threshold: {entry['threshold_kwh']} kWh)")
    print(f"    Multiplier    : x{entry['multiplier']}")
    print(f"    Priority      : {entry['priority']}")
    print(f"    Action       : {entry['action']}")

print("\n" + "=" * 72)

# -----
# 7. EXPORT SUMMARY FILE
# -----

def export_summary(sector_stats, plan, rejection_log, total_valid,
output_path):
    """Writes the full analysis, including the balancing plan, to a text
file."""
    with open(output_path, "w") as out:
        out.write("SMART GRID MUNICIPAL ENERGY ANALYTICS - SUMMARY REPORT\n")
        out.write(f"Generated: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n")
        out.write("=" * 72 + "\n\n")

        out.write(f"Valid records processed : {total_valid}\n")
        total_rejected = sum(rejection_log.values())
        out.write(f"Corrupted records rejected: {total_rejected}\n")
        for reason, count in rejection_log.items():
            out.write(f"    - {reason.replace('_', ' ').title():<28}:
{count}\n")

        out.write("\nSECTOR-WISE AGGREGATES\n")
        out.write("-" * 72 + "\n")
        for sector_name, stats in sector_stats.items():

            out.write(f"{sector_name}\n")
            out.write(f"    Readings      : {stats['reading_count']}\n")
            out.write(f"    Total Load    : {stats['total_load_kwh']:.2f}
kWh\n")

            out.write(f"    Average Load  : {stats['avg_load_kwh']:.2f}
kWh\n")

            out.write(f"    Peak Load     : {stats['peak_load_kwh']:.2f}
kWh\n")

            out.write(f"    Total Billing : Rs.
{stats['total_billing']:.2f}\n\n")

```

```

        out.write("PRIORITIZED BALANCING OPERATIONS PLAN\n")
        out.write("-" * 72 + "\n")
        for rank, entry in enumerate(plan, start=1):
            out.write(f"[{rank}] {entry['sector']} - Priority:
{entry['priority']}\n")
            out.write(f"    Avg Load: {entry['avg_load_kwh']} kWh | "
                    f"Threshold: {entry['threshold_kwh']} kWh | "
                    f"Multiplier: x{entry['multiplier']}\n")
            out.write(f"    Action: {entry['action']}\n\n")

# -----
# 8. MAIN / INTERACTIVE ENTRY POINT
# -----

def main():
    print("Smart Grid Municipal Energy Analytics and Peak Load Predictor")
    default_path = "grid_log.txt"
    user_path = input(
        f"Enter path to telemetry log file [default: {default_path}]: "
    ).strip()
    filepath = user_path if user_path else default_path

    if not os.path.isfile(filepath):
        print(f"Error: file '{filepath}' not found.")
        return

    valid_records, rejection_log = load_telemetry(filepath)
    sector_stats = aggregate_by_sector(valid_records)
    plan = generate_balancing_plan(sector_stats)

    print_dashboard(sector_stats, plan, rejection_log, len(valid_records))

    output_path = "balancing_operations_summary.txt"
    export_summary(sector_stats, plan, rejection_log, len(valid_records),
output_path)
    print(f"\nBalancing operations summary exported to: {output_path}")

if __name__ == "__main__":
    main()

```

Appendix B: Raw Telemetry Input Log (grid_log.txt)

This is the exact 25-line raw telemetry stream used as the test dataset for the results in Section 5. It contains 19 well-formed readings and 6 deliberately corrupted lines covering every rejection category the program is designed to catch.

```
RES#2026-09-01 00:00#412.5#6.20
COM#2026-09-01 00:00#1580.0#7.10
IND#2026-09-01 00:00#3210.0#5.80
RES#2026-09-01 01:00#388.2#6.10
COM#2026-09-01 01:00#1642.6#7.15
IND#2026-09-01 01:00#-150.0#5.75
RES#2026-09-01 02:00##6.05
COM#2026-09-01 02:00#1705.3#7.20
IND#2026-09-01 02:00#3540.8#5.90
XYZ#2026-09-01 03:00#500.0#6.00
RES#2026-09-01 03:00#455.7#6.25
COM#2026-09-01 03:00#1760.4#7.35
IND#2026-09-01 03:00#3620.0#6.05
RES#2026-09-01 04:00#421.9#6.15
COM#2026-09-01 04:00#1488.0
IND#2026-09-01 04:00#4005.5#6.10
RES#2026-09-01 05:00#560.3#6.40
COM#2026-09-01 05:00#1812.9#7.45
IND#2026-09-01 05:00#4210.2#6.20
RES#2026-09-01 06:00#601.4#6.55
COM#2026-09-01 06:00#1898.0#7.50
IND#2026-09-01 06:00#-2900.0#6.00
RES#2026-09-01 07:00#abc#6.30
COM#2026-09-01 07:00#1955.6#7.55
IND#2026-09-01 07:00#4320.7#6.25
```

Appendix C: Full Console Dashboard Output

This is the complete, unedited console output produced by running smart_grid_analytics.py against grid_log.txt, showing the rejection log, sector-wise aggregates, and the prioritized balancing operations plan exactly as printed by print_dashboard().

```
Smart Grid Municipal Energy Analytics and Peak Load Predictor
Enter path to telemetry log file [default: grid_log.txt]:
=====
SMART GRID MUNICIPAL ENERGY ANALYTICS - PEAK LOAD DASHBOARD
=====

Valid records processed : 19
Corrupted records rejected: 6
- Negative Or Invalid Value : 2
```

```

- Missing Numeric Field      : 1
- Misaligned Sector Key      : 1
- Malformed Field Count      : 1
- Non Numeric Value          : 1

```

```

-----
SECTOR          READINGS  TOTAL kWh    AVG kWh      PEAK kWh
-----
Residential     6           2840.00     473.33       601.40
Commercial      7           12354.80    1764.97      1955.60
Heavy_Industrial 6           22907.20    3817.87      4320.70

```

```

-----
PRIORITIZED BALANCING OPERATIONS PLAN
-----

```

```

[1] Sector      : Heavy_Industrial
    Avg Load    : 3817.87 kWh (Threshold: 3300.0 kWh)
    Multiplier   : x1.16
    Priority      : ELEVATED
    Action       : Apply dynamic tariff surcharge to Heavy_Industrial
                  consumers and issue an early demand-response advisory.

[2] Sector      : Commercial
    Avg Load    : 1764.97 kWh (Threshold: 1600.0 kWh)
    Multiplier   : x1.1
    Priority      : ELEVATED
    Action       : Apply dynamic tariff surcharge to Commercial consumers and
                  issue an early demand-response advisory.

[3] Sector      : Residential
    Avg Load    : 473.33 kWh (Threshold: 550.0 kWh)
    Multiplier   : x1.0
    Priority      : NORMAL
    Action       : Maintain standard distribution schedule for Residential.

```

```

=====
Balancing operations summary exported to: balancing_operations_summary.txt

```

Appendix D: Exported Summary File (balancing_operations_summary.txt)

This is the complete content of the text file written by export_summary() at the end of the run — the artifact a grid operator would actually receive and act on.

```

SMART GRID MUNICIPAL ENERGY ANALYTICS - SUMMARY REPORT
Generated: 2026-09-01 05:08:32
=====

Valid records processed : 19
Corrupted records rejected: 6
- Negative Or Invalid Value      : 2
- Missing Numeric Field          : 1
- Misaligned Sector Key          : 1
- Malformed Field Count          : 1
- Non Numeric Value              : 1

SECTOR-WISE AGGREGATES
-----

Residential
Readings      : 6
Total Load    : 2840.00 kWh
Average Load  : 473.33 kWh
Peak Load     : 601.40 kWh
Total Billing  : Rs. 17893.42

Commercial
Readings      : 7
Total Load    : 12354.80 kWh
Average Load  : 1764.97 kWh
Peak Load     : 1955.60 kWh
Total Billing  : Rs. 90685.58

Heavy_Industrial
Readings      : 6
Total Load    : 22907.20 kWh
Average Load  : 3817.87 kWh
Peak Load     : 4320.70 kWh
Total Billing  : Rs. 138950.89

PRIORITIZED BALANCING OPERATIONS PLAN
-----

[1] Heavy_Industrial - Priority: ELEVATED
Avg Load: 3817.87 kWh | Threshold: 3300.0 kWh | Multiplier: x1.16
Action: Apply dynamic tariff surcharge to Heavy_Industrial consumers and
       issue an early demand-response advisory.

[2] Commercial - Priority: ELEVATED
Avg Load: 1764.97 kWh | Threshold: 1600.0 kWh | Multiplier: x1.1

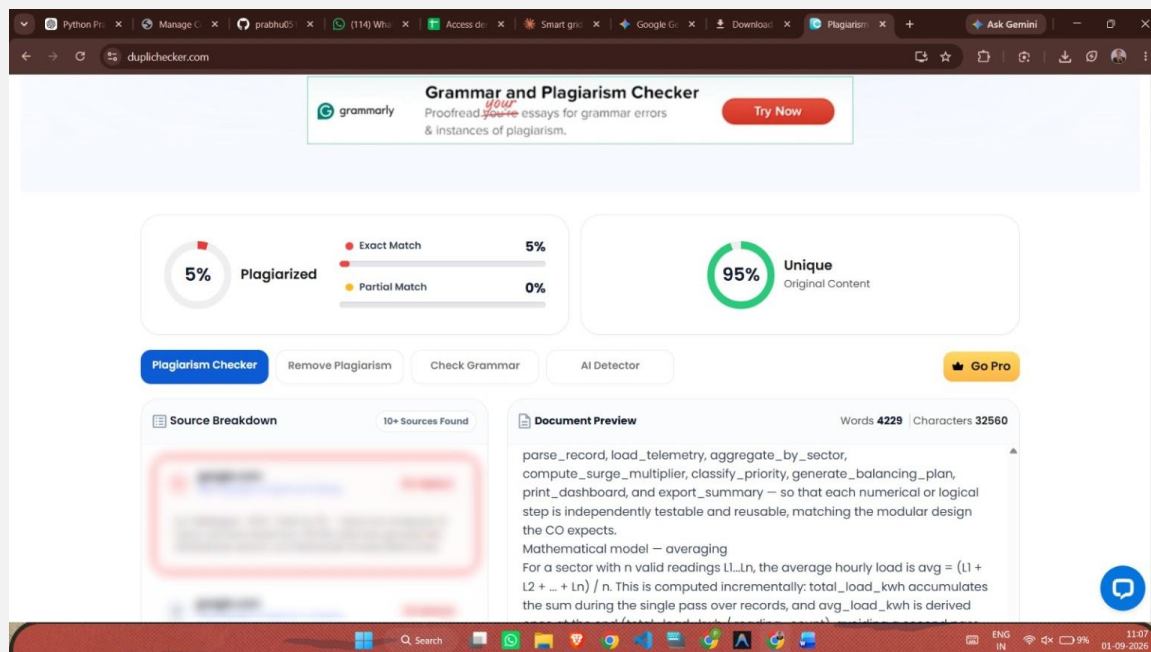
```

Action: Apply dynamic tariff surcharge to Commercial consumers and issue an early demand-response advisory.

[3] Residential - Priority: NORMAL

Avg Load: 473.33 kWh | Threshold: 550.0 kWh | Multiplier: x1.0

Action: Maintain standard distribution schedule for Residential.



<https://github.com/192424281simats-coder/CSA0812-PYTHON-SLOT-C>